PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING

10 Selected Case Studies

Prepared by	
Course	Python Programming
Assignment	Activity-Based Learning (ABL)
Selected case studies	1, 10, 19, 23, 30, 41, 59, 61, 74, 81



A structured student record containing problem statements, objectives, assumptions, algorithms, flowcharts, pseudocode, Python source code, test cases, sample output and conclusions.

Prepared with reference to the supplied case-study question bank and sample student presentation. Replace the student details above if needed and run each program before final submission.

CONTENTS & SUBMISSION NOTES

Selected case studies

Case No.	Title	Main concepts
1	University Scholarship Allocation	Conditions, lists, sorting, ranking
10	Campus Event Financial Planning	Arithmetic, costs, ceiling division
19	Multi-Product Shopping Budget Constraint	Lists, functions, while loop, budget
23	Student Timetable Conflict Detection	Tuples, nested loops, interval overlap
30	Student Email and Identifier Validation	Strings, slicing, validation
41	Nested List Classroom Occupancy Analysis	Nested lists, allocation, duplicate checks
59	Campus Transport Allocation	Ceiling division, capacity constraints
61	Nested List Examination Seating Validation	Nested lists, identifiers, validation
74	Campus Festival Registration and Financial Audit	Lists, duplicate detection, financial totals
81	Campus Event Seating and Entry Validation	Nested lists, ticket validation, capacity

Important note on assumptions

The original question bank states broad problem requirements but does not supply numerical policies, fees, thresholds, or sample datasets for every case. Where such values were needed, this record explicitly defines illustrative assumptions so that each program is executable and testable. These assumptions should be updated if your faculty provides different rules.

Output evidence

The terminal panels in this document are formatted sample runs for demonstration. For final submission, run each Python program in IDLE or VS Code and replace/augment these sample panels with screenshots captured from your own execution, as required by the circular.

Suggested repository structure

```
Python-Activity-Based-Learning/
|-- README.md
|-- Case_01_Scholarship_Allocation.py
|-- Case_10_Event_Financial_Planning.py
|-- Case_19_Shopping_Budget.py
|-- Case_23_Timetable_Conflict.py
|-- Case_30_Email_ID_Validation.py
|-- Case_41_Classroom_Occupancy.py
|-- Case_59_Transport_Allocation.py
|-- Case_61_Seating_Validation.py
|-- Case_74_Festival_Financial_Audit.py
|-- Case_81_Event_Entry_Validation.py
|-- report.pdf
|-- screenshots/
```

CASE STUDY 01 | University Scholarship Allocation

01 • DEFINE THE PROBLEM

Problem statement. Design a system that evaluates scholarship applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

Objectives

- Accept applicant details and academic/financial eligibility data.
- Apply all eligibility conditions consistently.
- Rank eligible applicants by aggregate marks, then attendance, then lower family income.
- Display eligible applicants in ranked order and identify those who do not qualify.

Assumptions and rules

- Aggregate and subject marks are percentages from 0 to 100.
- Applicants must score at least 60% aggregate and 50% in each listed subject.
- Attendance must be at least 75%; annual family income must not exceed ₹2,50,000.
- Tie-break order: higher aggregate, higher attendance, then lower income. Sample values are illustrative.

---

02 • ALGORITHM & FLOWCHART

Algorithm

1. Start and create an empty applicant list.
2. Read each applicant's name, aggregate, subject marks, attendance and income.
3. Validate numeric values and check all eligibility conditions.
4. Store eligible applicants with ranking fields.
5. Sort eligible applicants by aggregate descending, attendance descending and income ascending.
6. Display the ranked list and non-eligible count; stop.

Flowchart



Flow: Start → Input and validation → Process / decision → Display result → End.

CASE STUDY 01 | BUILD THE SOLUTION

03 • PSEUDOCODE

```
INPUT applicant records
eligible = empty list
FOR each applicant:
    IF marks, subject marks, attendance and income meet all rules:
        ADD applicant to eligible
SORT eligible BY aggregate descending,
    attendance descending, income ascending
DISPLAY eligible ranking
```

04 • PYTHON SOURCE CODE

The following program uses the assumptions stated in this case study. Run it with Python 3.

```
applicants = [
    ("Asha", 88, [82, 91, 86], 92, 180000),
    ("Bharat", 88, [79, 85, 90], 95, 160000),
    ("Charan", 72, [65, 70, 68], 78, 240000),
    ("Divya", 91, [92, 88, 94], 70, 150000),
    ("Esha", 84, [80, 48, 90], 90, 120000),
]

eligible = []
for name, aggregate, subjects, attendance, income in applicants:
    valid = (0 <= aggregate <= 100 and
             all(0 <= mark <= 100 for mark in subjects) and
             0 <= attendance <= 100 and income >= 0)
    if not valid:
        print(name, "- invalid record")
    elif (aggregate >= 60 and min(subjects) >= 50 and
          attendance >= 75 and income <= 250000):
        eligible.append((name, aggregate, attendance, income))
    else:
        print(name, "- not eligible")

eligible.sort(key=lambda x: (-x[1], -x[2], x[3]))
print("\nSCHOLARSHIP MERIT LIST")
for rank, (name, marks, attendance, income) in enumerate(eligible, 1):
    print(rank, name, "| Aggregate:", marks,
          "| Attendance:", attendance, "% | Income: ₹", income)
```

CASE STUDY 01 | TEST & VERIFY

05 • TEST CASES

Test input / condition	Expected result
Asha: 88%, all subjects ≥50, 92%, ₹1,80,000	Eligible; rank depends on other applicants
Divya: 91%, attendance 70%	Not eligible: attendance below 75%
Bharat and Asha: aggregate 88%	Bharat ranks higher due to attendance 95% vs 92%

06 • SAMPLE OUTPUT

Illustrative console output

```
SCHOLARSHIP MERIT LIST
1 Bharat | Aggregate: 88 | Attendance: 95% | Income: ₹ 160000
2 Asha | Aggregate: 88 | Attendance: 92% | Income: ₹ 180000
3 Charan | Aggregate: 72 | Attendance: 78% | Income: ₹ 240000
Divya - not eligible
Esha - not eligible
```

Output screenshot reminder

For the final repository, execute the program and add a genuine screenshot showing the inputs and output. The sample output above documents the expected behaviour but is not a screenshot of a local IDE run.

07 • CONCLUSION

The program applies multiple eligibility filters and uses a deterministic sorting key to produce a transparent merit list. Invalid or ineligible records are not included in the ranking.

CASE STUDY 10 | Campus Event Financial Planning

01 • DEFINE THE PROBLEM

Problem statement. Calculate venue, equipment, catering, transport and certificate costs; determine participant contributions, sponsorship shortfalls, and the minimum registration count needed to break even.

Objectives

- Calculate the total fixed and variable event expenditure.
- Calculate participant fee revenue and sponsorship contribution.
- Find the funding gap or surplus and minimum registrations required to break even.

Assumptions and rules

- Venue ₹12,000; equipment ₹5,000; transport ₹3,000; certificates ₹20 per participant; catering ₹150 per participant.
- Registration fee is ₹300 per participant and confirmed sponsorship is ₹10,000.
- Participants must be a non-negative integer; break-even registrations use ceiling division.

---

02 • ALGORITHM & FLOWCHART

Algorithm

1. Start and read participant count, registration fee and sponsorship.
2. Calculate fixed cost and variable cost per participant.
3. Calculate total cost, registration revenue and total available funds.
4. Compute surplus or shortfall.
5. Calculate the minimum participants needed for registration revenue plus sponsorship to cover fixed costs, allowing for per-person net contribution.
6. Display the financial summary and stop.

Flowchart



Flow: Start → Input and validation → Process / decision → Display result → End.

CASE STUDY 10 | BUILD THE SOLUTION

03 • PSEUDOCODE

```
INPUT participants, fee, sponsorship
fixed_cost = venue + equipment + transport
variable_cost = participants * (catering + certificate)
total_cost = fixed_cost + variable_cost
revenue = participants * fee
balance = revenue + sponsorship - total_cost
net_per_person = fee - catering - certificate
minimum = CEILING((fixed_cost - sponsorship) / net_per_person)
DISPLAY costs, revenue, balance, minimum
```

04 • PYTHON SOURCE CODE

The following program uses the assumptions stated in this case study. Run it with Python 3.

```
import math

participants = int(input("Number of participants: "))
fee = float(input("Registration fee per participant: ₹"))
sponsorship = float(input("Confirmed sponsorship: ₹"))

venue, equipment, transport = 12000, 5000, 3000
catering_per_person, certificate_per_person = 150, 20
fixed_cost = venue + equipment + transport
variable_per_person = catering_per_person + certificate_per_person

if participants < 0 or fee < 0 or sponsorship < 0:
    print("Inputs cannot be negative")
else:
    total_cost = fixed_cost + participants * variable_per_person
    revenue = participants * fee
    funds = revenue + sponsorship
    balance = funds - total_cost
    net_per_person = fee - variable_per_person

    print("Fixed cost: ₹", fixed_cost)
    print("Variable cost: ₹", participants * variable_per_person)
    print("Total expenditure: ₹", total_cost)
    print("Registration revenue: ₹", revenue)
    print("Total funds: ₹", funds)
    print("Final balance: ₹", balance)
    if net_per_person > 0:
        minimum = max(0, math.ceil((fixed_cost - sponsorship) /
                                   net_per_person))
        print("Minimum registrations to break even:", minimum)
    else:
        print("Break-even impossible at this fee: per-person contribution is not positive")
```

CASE STUDY 10 | TEST & VERIFY

05 • TEST CASES

Test input / condition	Expected result
Participants 100; fee ₹300; sponsorship ₹10,000	Cost ₹40,000; funds ₹40,000; balance ₹0
Participants 80; fee ₹300; sponsorship ₹10,000	Cost ₹36,400; funds ₹34,000; shortfall ₹2,400
Participants -2	Invalid input message

06 • SAMPLE OUTPUT

Illustrative console output

```
Number of participants: 100
Registration fee per participant: ₹300
Confirmed sponsorship: ₹10000
Fixed cost: ₹ 20000
Variable cost: ₹ 17000
Total expenditure: ₹ 37000
Registration revenue: ₹ 30000
Total funds: ₹ 40000
Final balance: ₹ 3000
Minimum registrations to break even: 70
```

Output screenshot reminder

For the final repository, execute the program and add a genuine screenshot showing the inputs and output. The sample output above documents the expected behaviour but is not a screenshot of a local IDE run.

07 • CONCLUSION

The program separates fixed costs from participant-dependent costs, compares funding against expenditure and estimates the minimum registration count under the stated pricing assumptions.

CASE STUDY 19 | Multi-Product Shopping Budget Constraint

01 • DEFINE THE PROBLEM

Problem statement. Evaluate a shopping basket containing multiple products and quantities, apply discount conditions, and determine which purchases must be reduced to remain within a fixed budget.

Objectives

- Store product names, unit prices and quantities in lists.
- Apply a basket discount when the subtotal reaches a defined threshold.
- Calculate the budget overrun and reduce quantities from lower-priority items until the basket fits the budget.

Assumptions and rules

- Illustrative budget is ₹2,000.
- A 10% discount applies when the pre-discount subtotal is at least ₹1,500.
- Products are reduced in the listed priority order, starting with the last item; quantities cannot become negative.
- The final discount is calculated on the reduced basket subtotal.

---

02 • ALGORITHM & FLOWCHART

Algorithm

1. Start and define the basket, quantities, prices and budget.
2. Calculate subtotal and applicable discount.
3. If total exceeds budget, reduce one unit at a time from the last product backwards, recalculating the bill after each reduction.
4. Stop when the bill fits the budget or no quantity remains.
5. Display original basket, reductions, final subtotal, discount and payable amount.

Flowchart



Flow: Start → Input and validation → Process / decision → Display result → End.

CASE STUDY 19 | BUILD THE SOLUTION

03 • PSEUDOCODE

```
INPUT budget and product basket
CALCULATE subtotal
IF subtotal >= discount_threshold:
    discount = subtotal * discount_rate
ELSE:
    discount = 0
WHILE subtotal - discount > budget AND any quantity > 0:
    reduce one unit from lowest-priority available item
    recalculate subtotal and discount
DISPLAY reductions and final payable amount
```

04 • PYTHON SOURCE CODE

The following program uses the assumptions stated in this case study. Run it with Python 3.

```
budget = 2000
discount_rate = 0.10
discount_threshold = 1500
products = ["Notebook", "Headphones", "Bottle", "Backpack"]
prices = [80, 650, 250, 900]
quantities = [4, 1, 2, 1]
original = quantities.copy()

def bill(qty):
    subtotal = sum(prices[i] * qty[i] for i in range(len(prices)))
    discount = subtotal * discount_rate if subtotal >= discount_threshold else 0
    return subtotal, discount, subtotal - discount

subtotal, discount, payable = bill(quantities)
print("Original payable: ₹", round(payable, 2))

i = len(quantities) - 1
while payable > budget and i >= 0:
    if quantities[i] > 0:
        quantities[i] -= 1
        subtotal, discount, payable = bill(quantities)
    else:
        i -= 1

print("\nPURCHASE ADJUSTMENT")
for i in range(len(products)):
    reduced = original[i] - quantities[i]
    if reduced > 0:
        print(products[i], "- reduce by", reduced, "unit(s)")
print("Final subtotal: ₹", round(subtotal, 2))
print("Discount: ₹", round(discount, 2))
print("Final payable: ₹", round(payable, 2))
print("Budget remaining: ₹", round(budget - payable, 2))
if payable > budget:
    print("Basket cannot fit the budget even after removing all items")
```

CASE STUDY 19 | TEST & VERIFY

05 • TEST CASES

Test input / condition	Expected result
Budget ₹2,000; original payable above ₹2,000	Program reduces lower-priority products until payable ≤ ₹2,000
Subtotal ₹1,400	No discount; discount ₹0
Subtotal ₹1,600	10% discount; discount ₹160

06 • SAMPLE OUTPUT

Illustrative console output

```
Original payable: ₹ 2430.0
PURCHASE ADJUSTMENT
Backpack - reduce by 1 unit(s)
Final subtotal: ₹ 1580
Discount: ₹ 158.0
Final payable: ₹ 1422.0
Budget remaining: ₹ 578.0
```

Output screenshot reminder

For the final repository, execute the program and add a genuine screenshot showing the inputs and output. The sample output above documents the expected behaviour but is not a screenshot of a local IDE run.

07 • CONCLUSION

The solution uses lists, a function and a while loop to repeatedly recalculate the basket and enforce a spending limit. The reduction order is an explicit assumption and can be changed to match a real shopping priority.

CASE STUDY 23 | Student Timetable Conflict Detection

01 • DEFINE THE PROBLEM

Problem statement. Compare course schedules represented by strings and tuples, detect overlapping periods, identify unavailable rooms, and validate a proposed timetable.

Objectives

- Represent timetable entries as tuples containing course, day, start, end and room.
- Detect time overlaps for courses scheduled on the same day.
- Detect room clashes when two classes use the same room at overlapping times.

Assumptions and rules

- Time is represented as integer minutes after midnight.
- A class occupies the interval start ≤ time < end; classes ending exactly when another starts do not conflict.
- Room availability is checked against the supplied room schedule.
- The dataset is illustrative.

---

02 • ALGORITHM & FLOWCHART

Algorithm

1. Start and define course tuples and unavailable-room tuples.
2. Validate that each interval has start time earlier than end time.
3. Compare every pair of classes on the same day.
4. If intervals overlap, report a student timetable conflict; if the room is the same, report a room clash.
5. Compare each class against unavailable-room intervals.
6. Display all conflicts and validation status.

Flowchart



Flow: Start → Input and validation → Process / decision → Display result → End.

CASE STUDY 23 | BUILD THE SOLUTION

03 • PSEUDOCODE

```
FOR each class:
    IF start >= end: report invalid interval
FOR each pair of classes:
    IF same day AND start1 < end2 AND start2 < end1:
        report timetable overlap
        IF same room: report room clash
FOR each class and unavailable room interval:
    IF same day, same room and intervals overlap:
        report room unavailable
DISPLAY all conflicts
```

04 • PYTHON SOURCE CODE

The following program uses the assumptions stated in this case study. Run it with Python 3.

```
# Each class: (course, day, start_minute, end_minute, room)
classes = [
    ("C Programming", "Mon", 540, 600, "R101"),
    ("Mathematics", "Mon", 570, 630, "R102"),
    ("Physics Lab", "Mon", 630, 690, "LAB1"),
    ("Web Technology", "Tue", 540, 600, "R101"),
]
# Room unavailable: (day, start_minute, end_minute, room)
unavailable = [("Mon", 630, 660, "LAB1")]

def overlaps(start1, end1, start2, end2):
    return start1 < end2 and start2 < end1

conflicts = 0
for course, day, start, end, room in classes:
    if start >= end:
        print("INVALID INTERVAL:", course)
        conflicts += 1

for i in range(len(classes)):
    c1, d1, s1, e1, r1 = classes[i]
    for j in range(i + 1, len(classes)):
        c2, d2, s2, e2, r2 = classes[j]
        if d1 == d2 and overlaps(s1, e1, s2, e2):
            print("TIMETABLE CLASH:", c1, "and", c2, "on", d1)
            if r1 == r2:
                print("ROOM CLASH:", r1)
            conflicts += 1

for course, day, start, end, room in classes:
    for uday, us, ue, uroom in unavailable:
        if day == uday and room == uroom and overlaps(start, end, us, ue):
            print("ROOM UNAVAILABLE:", course, "in", room)
            conflicts += 1

if conflicts == 0:
    print("Timetable is valid")
else:
    print("Total issues found:", conflicts)
```

CASE STUDY 23 | TEST & VERIFY

05 • TEST CASES

Test input / condition	Expected result
C Programming 09:00–10:00; Mathematics 09:30–10:30, same day	Timetable clash reported
Two classes 09:00–10:00 and 10:00–11:00	No overlap
Physics Lab 10:30–11:30 in LAB1; LAB1 unavailable 10:30–11:00	Room unavailable reported

06 • SAMPLE OUTPUT

Illustrative console output

```
TIMETABLE CLASH: C Programming and Mathematics on Mon
ROOM UNAVAILABLE: Physics Lab in LAB1
Total issues found: 2
```

Output screenshot reminder

For the final repository, execute the program and add a genuine screenshot showing the inputs and output. The sample output above documents the expected behaviour but is not a screenshot of a local IDE run.

07 • CONCLUSION

The program checks half-open time intervals using tuple records and nested loops. It identifies both course overlaps and room unavailability without treating back-to-back classes as clashes.

CASE STUDY 30 | Student Email and Identifier Validation

01 • DEFINE THE PROBLEM

Problem statement. Validate student email-like strings and registration identifiers against supplied formatting rules, reporting malformed separators, invalid character groups, and missing components.

Objectives

- Check email-like strings for one @ symbol, a non-empty username and domain, and a dot in the domain.
- Validate registration IDs using a defined pattern.
- Explain invalid records rather than returning only a pass/fail result.

Assumptions and rules

- Email rule: exactly one @; username and domain must be non-empty; domain must contain a dot not at either end; no spaces are allowed.
- Registration ID format: SRM followed by 2 digits, a hyphen, 2 uppercase letters, a hyphen and 4 digits (example SRM26-CS-1042).
- These are classroom validation rules, not full standards-compliant email validation.

---

02 • ALGORITHM & FLOWCHART

Algorithm

1. Start and read an email-like string and registration ID.
2. Check spaces and count the @ symbol.
3. Split the email at @ and validate username and domain components.
4. Check the registration ID against the specified character positions and lengths.
5. Collect and print every validation error; otherwise report valid input.

Flowchart



Flow: Start → Input and validation → Process / decision → Display result → End.

CASE STUDY 30 | BUILD THE SOLUTION

03 • PSEUDOCODE

```
INPUT email, registration_id
errors = empty list
CHECK email has no spaces and exactly one @
SPLIT email into username and domain
CHECK username is non-empty
CHECK domain has a dot and valid endpoints
CHECK registration_id matches SRMdd-LL-dddd
IF errors is empty: PRINT valid
ELSE: PRINT each error
```

04 • PYTHON SOURCE CODE

The following program uses the assumptions stated in this case study. Run it with Python 3.

```
def validate_email(email):
    errors = []
    if " " in email:
        errors.append("Email must not contain spaces")
    if email.count("@") != 1:
        errors.append("Email must contain exactly one @")
    else:
        username, domain = email.split("@")
        if not username:
            errors.append("Username is missing")
        if not domain:
            errors.append("Domain is missing")
        elif "." not in domain or domain.startswith(".") or domain.endswith("."):
            errors.append("Domain must contain a dot between characters")
    return errors

def validate_id(reg_id):
    # Required format: SRM26-CS-1042
    if len(reg_id) != 13:
        return False
    return (reg_id[:3] == "SRM" and reg_id[3:5].isdigit()
            and reg_id[5] == "-" and reg_id[6:8].isalpha()
            and reg_id[6:8].isupper() and reg_id[8] == "-"
            and reg_id[9:13].isdigit())

email = input("Student email: ").strip()
reg_id = input("Registration ID: ").strip()

email_errors = validate_email(email)
print("\nEMAIL CHECK")
if email_errors:
    for error in email_errors:
        print("-", error)
else:
    print("Email format is valid")

print("\nIDENTIFIER CHECK")
if validate_id(reg_id):
    print("Registration ID is valid")
else:
    print("Registration ID is invalid; expected format SRM26-CS-1042")
```

CASE STUDY 30 | TEST & VERIFY

05 • TEST CASES

Test input / condition	Expected result
sahith@example.com; SRM26-CS-1042	Email valid; identifier valid
sahithexample.com; SRM26-CS-1042	Email invalid: exactly one @ required
sahith@domain; srm26-cs-1042	Email invalid: domain needs dot; ID invalid

06 • SAMPLE OUTPUT

Illustrative console output

```
Student email: sahith@example.com
Registration ID: SRM26-CS-1042
EMAIL CHECK
Email format is valid
IDENTIFIER CHECK
Registration ID is valid
```

Output screenshot reminder

For the final repository, execute the program and add a genuine screenshot showing the inputs and output. The sample output above documents the expected behaviour but is not a screenshot of a local IDE run.

07 • CONCLUSION

The program uses string operations, slicing and character checks to validate structured text. Its rules are deliberately limited and explicit, making it suitable for a syllabus-level assignment.

CASE STUDY 41 | Nested List Classroom Occupancy Analysis

01 • DEFINE THE PROBLEM

Problem statement. Represent multiple classrooms and seating positions as nested lists; allocate students, preserve reserved seats, detect duplicate assignments, and calculate vacant capacity.

Objectives

- Represent classrooms as a nested list of seats.
- Preserve reserved positions and allocate students only to empty, non-reserved seats.
- Detect duplicate student assignments and calculate vacant seats per classroom.

Assumptions and rules

- Each classroom has a fixed number of seats in a list.
- The string 'RESERVED' marks a seat unavailable for general allocation; None means vacant.
- A student roll number must not appear more than once across all classrooms.
- Allocation follows classroom and seat order.

---

02 • ALGORITHM & FLOWCHART

Algorithm

1. Start and create nested lists for classrooms.
2. Mark reserved seats and place any existing allocations.
3. For each requested student, check for duplicate roll number.
4. Find the first None seat that is not reserved and assign the student.
5. If no seat is available, report unallocated student.
6. Count occupied, reserved and vacant seats for each classroom.

Flowchart



Flow: Start → Input and validation → Process / decision → Display result → End.

CASE STUDY 41 | BUILD THE SOLUTION

03 • PSEUDOCODE

```
FOR each student in waiting_list:
    IF student already appears in any classroom:
        report duplicate
    ELSE:
        find first None seat in classroom order
        IF found: assign student
        ELSE: report no vacant seat
FOR each classroom:
    count vacant None seats and occupied seats
DISPLAY seating matrix and counts
```

04 • PYTHON SOURCE CODE

The following program uses the assumptions stated in this case study. Run it with Python 3.

```
classrooms = [
    ["RESERVED", None, None, "S101"],
    [None, "RESERVED", None, None],
    ["S105", None, None, None],
]
waiting_list = ["S102", "S103", "S101", "S104", "S106"]

def all_students(room_list):
    return [seat for room in room_list
            for seat in room if seat is not None and seat != "RESERVED"]

for student in waiting_list:
    if student in all_students(classrooms):
        print(student, "- duplicate assignment rejected")
        continue
    placed = False
    for room in classrooms:
        for seat_index in range(len(room)):
            if room[seat_index] is None:
                room[seat_index] = student
                print(student, "assigned to classroom",
                      classrooms.index(room) + 1,
                      "seat", seat_index + 1)
                placed = True
                break
        if placed:
            break
    if not placed:
        print(student, "- no vacant seat")

print("\nCLASSROOM SUMMARY")
for i, room in enumerate(classrooms, 1):
    vacant = room.count(None)
    reserved = room.count("RESERVED")
    occupied = len(room) - vacant - reserved
    print("Classroom", i, "| Occupied:", occupied,
          "| Reserved:", reserved, "| Vacant:", vacant)
    print(room)
```

CASE STUDY 41 | TEST & VERIFY

05 • TEST CASES

Test input / condition	Expected result
S101 already assigned; S101 requested again	Duplicate assignment rejected
Classroom has None seat	Next student is assigned to first available seat
All non-reserved seats full	Student is reported as having no vacant seat

06 • SAMPLE OUTPUT

Illustrative console output

```
S102 assigned to classroom 1 seat 2
S103 assigned to classroom 1 seat 3
S101 - duplicate assignment rejected
S104 assigned to classroom 2 seat 1
S106 assigned to classroom 2 seat 3
CLASSROOM SUMMARY
Classroom 1 | Occupied: 3 | Reserved: 1 | Vacant: 0
Classroom 2 | Occupied: 2 | Reserved: 1 | Vacant: 1
Classroom 3 | Occupied: 1 | Reserved: 0 | Vacant: 3
```

Output screenshot reminder

For the final repository, execute the program and add a genuine screenshot showing the inputs and output. The sample output above documents the expected behaviour but is not a screenshot of a local IDE run.

07 • CONCLUSION

Nested lists model classroom and seat structure directly. The program prevents duplicate roll numbers, preserves reserved seats and reports occupied and vacant capacity.

CASE STUDY 59 | Campus Transport Allocation

01 • DEFINE THE PROBLEM

Problem statement. Determine the number of buses needed for several routes, account for bus capacities and passenger demand, and identify unserved passengers.

Objectives

- Store route names, passenger demand and bus capacity.
- Calculate required buses using ceiling division.
- Calculate total seats provided and any unserved passengers when a route has a bus limit.

Assumptions and rules

- Each bus has a fixed capacity for its route.
- The campus has a maximum number of buses available per route, shown in the data.
- Buses are allocated in whole numbers; passengers cannot be split into fractional seats.
- Unserved passengers are demand minus allocated capacity, limited to zero.

---

02 • ALGORITHM & FLOWCHART

Algorithm

1. Start and read route demand, bus capacity and maximum buses.
2. Validate non-negative demand and positive capacity.
3. Calculate buses required using ceiling division.
4. Allocate the smaller of required buses and available buses.
5. Calculate served capacity and unserved passengers.
6. Display route-wise allocation and totals.

Flowchart



Flow: Start → Input and validation → Process / decision → Display result → End.

CASE STUDY 59 | BUILD THE SOLUTION

03 • PSEUDOCODE

```
FOR each route:
    IF demand < 0 OR capacity <= 0: report invalid
    required = CEILING(demand / capacity)
    allocated = MIN(required, buses_available)
    seats = allocated * capacity
    unserved = MAX(0, demand - seats)
    DISPLAY route, required, allocated, unserved
DISPLAY overall totals
```

04 • PYTHON SOURCE CODE

The following program uses the assumptions stated in this case study. Run it with Python 3.

```
import math

routes = [
    ("Hostel Route", 135, 50, 3),
    ("City Route", 92, 40, 2),
    ("Railway Station", 48, 30, 2),
    ("Airport Route", 20, 25, 1),
]
total_demand = total_buses = total_unserved = 0

print("CAMPUS TRANSPORT ALLOCATION")
for route, demand, capacity, available in routes:
    if demand < 0 or capacity <= 0 or available < 0:
        print(route, "- invalid route data")
        continue
    required = math.ceil(demand / capacity)
    allocated = min(required, available)
    seats = allocated * capacity
    unserved = max(0, demand - seats)
    total_demand += demand
    total_buses += allocated
    total_unserved += unserved
    print(route, "| Demand:", demand,
          "| Buses needed:", required,
          "| Buses allocated:", allocated,
          "| Unserved:", unserved)

print("Total passengers:", total_demand)
print("Total buses allocated:", total_buses)
print("Total unserved passengers:", total_unserved)
```

CASE STUDY 59 | TEST & VERIFY

05 • TEST CASES

Test input / condition	Expected result
Demand 135; capacity 50; 3 buses available	3 buses; 15 seats of spare capacity
Demand 92; capacity 40; 2 buses available	2 buses; 12 passengers unserved
Demand 0; capacity 30; 2 buses available	0 buses required; 0 unserved

06 • SAMPLE OUTPUT

Illustrative console output

```
CAMPUS TRANSPORT ALLOCATION
Hostel Route | Demand: 135 | Buses needed: 3 | Buses allocated: 3 | Unserved: 0
City Route | Demand: 92 | Buses needed: 3 | Buses allocated: 2 | Unserved: 12
Railway Station | Demand: 48 | Buses needed: 2 | Buses allocated: 2 | Unserved: 0
Airport Route | Demand: 20 | Buses needed: 1 | Buses allocated: 1 | Unserved: 0
Total passengers: 295
Total buses allocated: 8
Total unserved passengers: 12
```

Output screenshot reminder

For the final repository, execute the program and add a genuine screenshot showing the inputs and output. The sample output above documents the expected behaviour but is not a screenshot of a local IDE run.

07 • CONCLUSION

The program uses ceiling division to estimate buses and respects the fleet limit on each route. It identifies routes where available transport capacity is insufficient.

CASE STUDY 61 | Nested List Examination Seating Validation

01 • DEFINE THE PROBLEM

Problem statement. Validate a complete seating matrix for duplicate roll numbers, reserved positions, empty seats, and invalid student allocations.

Objectives

- Represent the examination hall as a nested list.
- Check duplicate roll numbers and reserved-seat misuse.
- Count empty seats and report invalid allocations.

Assumptions and rules

- None represents an empty seat; 'RESERVED' represents a seat unavailable for student allocation.
- Valid roll numbers begin with 'S' followed by digits, such as S101.
- A roll number may appear only once in the entire matrix.
- The sample matrix intentionally includes errors for validation.

---

02 • ALGORITHM & FLOWCHART

Algorithm

1. Start and define the seating matrix.
2. Traverse every row and seat, recording student positions.
3. Report empty seats and reserved positions.
4. Validate roll-number format and detect duplicate roll numbers.
5. Display the number of valid students, empty seats, reserved seats and issues.

Flowchart



Flow: Start → Input and validation → Process / decision → Display result → End.

CASE STUDY 61 | BUILD THE SOLUTION

03 • PSEUDOCODE

```
seen = empty list
FOR each row and seat:
    IF seat is None: count empty
    ELSE IF seat is RESERVED: count reserved
    ELSE:
        IF roll number has invalid format: report invalid
        IF roll number in seen: report duplicate
        ELSE: add roll number to seen
DISPLAY counts and validation status
```

04 • PYTHON SOURCE CODE

The following program uses the assumptions stated in this case study. Run it with Python 3.

```
seating = [
    ["S101", "S102", None, "RESERVED"],
    ["S103", "S102", "S105", None],
    ["S106", "BAD7", "RESERVED", "S108"],
]

seen = []
issues = 0
empty_seats = reserved_seats = valid_students = 0

for row_index, row in enumerate(seating, 1):
    for seat_index, value in enumerate(row, 1):
        location = "row " + str(row_index) + ", seat " + str(seat_index)
        if value is None:
            empty_seats += 1
            print("EMPTY SEAT:", location)
        elif value == "RESERVED":
            reserved_seats += 1
        else:
            valid_id = (len(value) >= 2 and value[0] == "S"
                        and value[1:].isdigit())
            if not valid_id:
                print("INVALID ROLL NUMBER:", value, "at", location)
                issues += 1
            elif value in seen:
                print("DUPLICATE ROLL NUMBER:", value, "at", location)
                issues += 1
            else:
                seen.append(value)
                valid_students += 1

print("\nSEATING VALIDATION SUMMARY")
print("Valid student allocations:", valid_students)
print("Empty seats:", empty_seats)
print("Reserved seats:", reserved_seats)
print("Issues found:", issues)
print("Status:", "VALID" if issues == 0 else "CHECK REQUIRED")
```

CASE STUDY 61 | TEST & VERIFY

05 • TEST CASES

Test input / condition	Expected result
S102 appears twice	Second occurrence is flagged as duplicate
Seat value None	Counted as an empty seat
Roll number BAD7	Flagged as invalid because it does not start with S followed by digits

06 • SAMPLE OUTPUT

Illustrative console output

```
EMPTY SEAT: row 1, seat 3
DUPLICATE ROLL NUMBER: S102 at row 2, seat 2
EMPTY SEAT: row 2, seat 4
INVALID ROLL NUMBER: BAD7 at row 3, seat 2
SEATING VALIDATION SUMMARY
Valid student allocations: 6
Empty seats: 2
Reserved seats: 2
Issues found: 2
Status: CHECK REQUIRED
```

Output screenshot reminder

For the final repository, execute the program and add a genuine screenshot showing the inputs and output. The sample output above documents the expected behaviour but is not a screenshot of a local IDE run.

07 • CONCLUSION

The program validates a nested seating matrix by tracking previously seen roll numbers and classifying each seat as occupied, empty or reserved. It reports both duplicate assignments and malformed identifiers.

CASE STUDY 74 | Campus Festival Registration and Financial Audit

01 • DEFINE THE PROBLEM

Problem statement. Reconcile participant lists, registration fees, sponsorship contributions and expenses; detect duplicate registrations and calculate the final financial balance.

Objectives

- Identify duplicate participant registration IDs.
- Calculate valid registration revenue and sponsorship total.
- Compare total funds with event expenses and report surplus or deficit.

Assumptions and rules

- Each valid registration ID is unique and each registration fee is ₹250.
- Only the first occurrence of a registration ID is counted; duplicates are rejected.
- Sponsorships are listed as non-negative amounts; expenses are non-negative.
- All values are illustrative and in Indian rupees.

---

02 • ALGORITHM & FLOWCHART

Algorithm

1. Start and define participant records, sponsorships and expense records.
2. Traverse participants and reject duplicate registration IDs.
3. Sum fees for unique participants.
4. Sum sponsorship contributions and event expenses.
5. Calculate total funds and final balance.
6. Display duplicates, revenue, sponsorship, expenditure and surplus/deficit.

Flowchart



Flow: Start → Input and validation → Process / decision → Display result → End.

CASE STUDY 74 | BUILD THE SOLUTION

03 • PSEUDOCODE

```
seen_ids = empty list
valid_count = 0
FOR each participant:
    IF registration ID already in seen_ids:
        report duplicate
    ELSE:
        add ID to seen_ids
        valid_count = valid_count + 1
registration_revenue = valid_count * fee
funds = registration_revenue + SUM(sponsorships)
balance = funds - SUM(expenses)
DISPLAY audit summary
```

04 • PYTHON SOURCE CODE

The following program uses the assumptions stated in this case study. Run it with Python 3.

```
participants = [
    ("REG101", "Asha"), ("REG102", "Bharat"),
    ("REG103", "Charan"), ("REG102", "Bharat Duplicate"),
    ("REG104", "Divya"),
]
fee = 250
sponsorships = [10000, 7500, 5000]
expenses = [12000, 8000, 4500, 3000]

seen_ids = []
valid_count = 0
for reg_id, name in participants:
    if reg_id in seen_ids:
        print("DUPLICATE REGISTRATION REJECTED:", reg_id, name)
    else:
        seen_ids.append(reg_id)
        valid_count += 1

registration_revenue = valid_count * fee
sponsorship_total = sum(sponsorships)
expense_total = sum(expenses)
total_funds = registration_revenue + sponsorship_total
balance = total_funds - expense_total

print("\nFESTIVAL FINANCIAL AUDIT")
print("Valid participants:", valid_count)
print("Registration revenue: ₹", registration_revenue)
print("Sponsorship total: ₹", sponsorship_total)
print("Total funds: ₹", total_funds)
print("Total expenses: ₹", expense_total)
print("Final balance: ₹", balance)
if balance > 0:
    print("Result: SURPLUS")
elif balance < 0:
    print("Result: DEFICIT")
else:
    print("Result: BREAK-EVEN")
```

CASE STUDY 74 | TEST & VERIFY

05 • TEST CASES

Test input / condition	Expected result
5 entries with one duplicate ID; fee ₹250	4 valid registrations; revenue ₹1,000
Sponsorships ₹10,000 + ₹7,500 + ₹5,000	Sponsorship total ₹22,500
Funds ₹23,500; expenses ₹27,500	Deficit ₹4,000

06 • SAMPLE OUTPUT

Illustrative console output

```
DUPLICATE REGISTRATION REJECTED: REG102 Bharat Duplicate
FESTIVAL FINANCIAL AUDIT
Valid participants: 4
Registration revenue: ₹ 1000
Sponsorship total: ₹ 22500
Total funds: ₹ 23500
Total expenses: ₹ 27500
Final balance: ₹ -4000
Result: DEFICIT
```

Output screenshot reminder

For the final repository, execute the program and add a genuine screenshot showing the inputs and output. The sample output above documents the expected behaviour but is not a screenshot of a local IDE run.

07 • CONCLUSION

The program reconciles participant registrations and event finances. It prevents duplicate IDs from inflating revenue and gives a clear surplus or deficit figure for the festival.

CASE STUDY 81 | Campus Event Seating and Entry Validation

01 • DEFINE THE PROBLEM

Problem statement. Generate seating assignments from nested lists, validate tickets against registration identifiers, detect duplicate entries, and report available seating capacity.

Objectives

- Allocate registered participants to seats represented by nested lists.
- Validate ticket IDs against the registration list.
- Reject duplicate entries and calculate remaining seat capacity.

Assumptions and rules

- Each registered ticket ID may enter only once.
- Seats are assigned in row and seat order; None denotes an available seat.
- A ticket must be present in the registered ticket list to be accepted.
- The system is a classroom simulation and does not use barcode scanning.

---

02 • ALGORITHM & FLOWCHART

Algorithm

1. Start and define registered ticket IDs, entry attempts and seating matrix.
2. For each ticket attempt, reject unknown tickets and previously used tickets.
3. For a valid first-time ticket, allocate the next available seat.
4. If no seat is available, report capacity full.
5. Count used and vacant seats and display the entry summary.

Flowchart



Flow: Start → Input and validation → Process / decision → Display result → End.

CASE STUDY 81 | BUILD THE SOLUTION

03 • PSEUDOCODE

```
used_tickets = empty list
FOR each ticket in entry_attempts:
    IF ticket not registered: reject
    ELSE IF ticket in used_tickets: reject duplicate entry
    ELSE:
        find first empty seat
        IF seat exists:
            assign ticket and add to used_tickets
        ELSE: report capacity full
COUNT occupied and vacant seats
DISPLAY entry summary
```

04 • PYTHON SOURCE CODE

The following program uses the assumptions stated in this case study. Run it with Python 3.

```
registered_tickets = ["T101", "T102", "T103", "T104", "T105"]
entry_attempts = ["T101", "T102", "T102", "T999", "T103", "T104"]
seats = [[None, None, None], [None, None, None]]

used_tickets = []
accepted = rejected = 0

for ticket in entry_attempts:
    if ticket not in registered_tickets:
        print(ticket, "- REJECTED: ticket not registered")
        rejected += 1
        continue
    if ticket in used_tickets:
        print(ticket, "- REJECTED: duplicate entry")
        rejected += 1
        continue

    assigned = False
    for row in seats:
        for i in range(len(row)):
            if row[i] is None:
                row[i] = ticket
                used_tickets.append(ticket)
                accepted += 1
                print(ticket, "- ACCEPTED; seat assigned")
                assigned = True
                break
        if assigned:
            break
    if not assigned:
        print(ticket, "- REJECTED: seating capacity full")
        rejected += 1

vacant = sum(row.count(None) for row in seats)
print("\nEVENT ENTRY SUMMARY")
print("Accepted entries:", accepted)
print("Rejected entries:", rejected)
print("Occupied seats:", accepted)
print("Available seats:", vacant)
print("Seating matrix:", seats)
```

CASE STUDY 81 | TEST & VERIFY

05 • TEST CASES

Test input / condition	Expected result
T101 is registered and not previously used	Accepted and assigned a seat
T102 appears twice in entry attempts	Second entry rejected as duplicate
T999 not in registered list	Rejected as unregistered ticket

06 • SAMPLE OUTPUT

Illustrative console output

```
T101 - ACCEPTED; seat assigned
T102 - ACCEPTED; seat assigned
T102 - REJECTED: duplicate entry
T999 - REJECTED: ticket not registered
T103 - ACCEPTED; seat assigned
T104 - ACCEPTED; seat assigned
EVENT ENTRY SUMMARY
Accepted entries: 4
Rejected entries: 2
Occupied seats: 4
Available seats: 2
```

Output screenshot reminder

For the final repository, execute the program and add a genuine screenshot showing the inputs and output. The sample output above documents the expected behaviour but is not a screenshot of a local IDE run.

07 • CONCLUSION

The solution combines a registration list with nested seat lists to validate ticket status, prevent repeat entry and monitor venue capacity in a transparent way.

FINAL SUBMISSION CHECKLIST

- Enter the correct student name, roll number, branch and section on the cover page.
- Save each program as a separate .py file using the case number and a clear title.
- Run every program and verify the results against the test cases.
- Capture actual output screenshots from IDLE, VS Code or another Python environment.
- Ensure code indentation, spelling, flowcharts and expected outputs are correct.
- Create the GitHub repository named Python-Activity-Based-Learning.
- Upload all ten programs, this report, supporting documentation and output screenshots.
- Add a README.md with student details and the titles of all selected case studies.
- Submit the repository URL to the Python Programming faculty member / Lab Mentor.

REFERENCE

Case-study titles and mandatory deliverables are based on the supplied “Python Activity-Based Learning” question bank. Layout and section organization are modelled on the supplied sample student presentation. Numerical values and datasets are illustrative wherever the question bank leaves them unspecified.